ALARM CLOCK SYSTEM

Team ID: Team-21 | Section: 9 | Guide: Mrs Y. Harika Devi

T. Karthik – 2520030234 | CH. Manikanta – 2520030239 | T. Jaswanth Reddy – 2520030284

Operating Systems and Systems Programming (OSSP) | KLH Bachupally Campus, CSE | PBL 2026–27

Abstract

- A multi-alarm clock system to set and trigger alarms using Linux/C processes, signals and timers.

Outcome: a reliable alarm that rings on time and handles invalid input.

Problem Statement & Objectives

- Problem: Alarms must fire on time without blocking the program, even with several alarms set.

Objective 1: Set, list and delete alarms safely
Objective 2: Fire alarms on time using signals
Objective 3: Ring and snooze via child processes
Scope: Single-user alarm clock on Linux covering processes, signals and file I/O.

Existing vs Proposed System

Aspect	Basic / Existing Approach	Proposed Project
Process handling	One blocking loop; one alarm at a time	fork() child processes ring and snooze
Communication	No link between timer and ringer	Signals (SIGALRM, SIGCHLD) and pipe()
Resource handling	Alarms lost when program exits	Alarms stored in a file and reloaded
Error handling	Invalid times crash or are ignored	Input validation and error codes
Key gap	Cannot manage many alarms reliably	Multi-alarm scheduling using OS services

Feasibility Study

- Technical: C, Linux/Ubuntu, GCC and standard system calls (alarm, fork, signal).

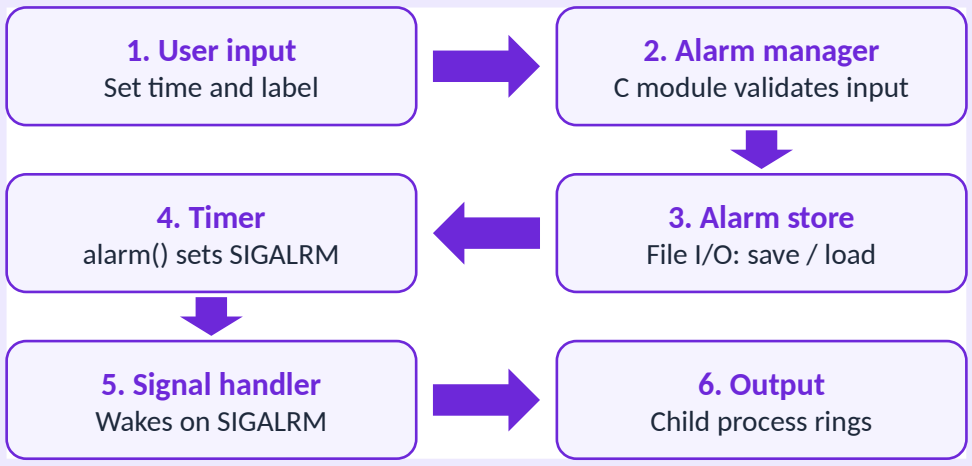
Economic: All tools are open-source; no hardware or hosting cost.
Operational: Easy to use; runs on any Linux/WSL machine without special permissions.
Schedule: Planned, built and tested within the semester timeline.

Tools & Technologies

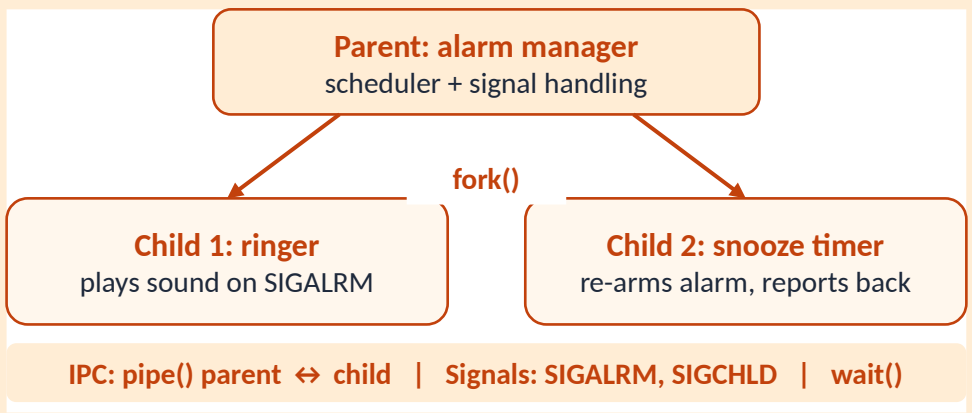
- Programming: C (core logic); HTML/CSS/JS web UI

Platform: Linux / Ubuntu (WSL or native)
OS Concepts: Processes, signals, timers, file handling
System Calls / APIs: fork(), alarm(), sleep(), signal(), wait(), pipe()
Tools: GCC, Make, GDB, Git & GitHub, VS Code

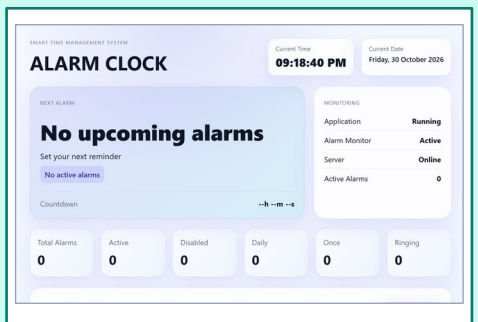
System Workflow / Architecture



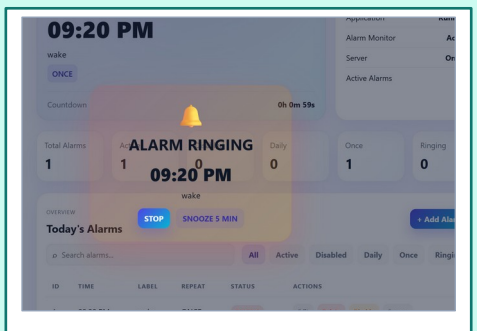
Process, IPC & Resource Design



Implementation Screenshots



Dashboard view



Alarm ringing: Stop / Snooze

Results & Testing

- Sample: a 07:00 alarm rings on time; snooze re-arms it.

Tested: valid/invalid times, multiple alarms.

Conclusion

- Built an alarm clock using Linux/C processes and signals.

Meets the objectives; shows core OSSP concepts.

Future Scope

- Enhancement 1: Recurring alarms and custom ringtones.

Enhancement 2: Automated tests and cross-device sync.

References: [1] Linux man-pages [2] Operating Systems textbook / course material [3] GCC / GNU documentation